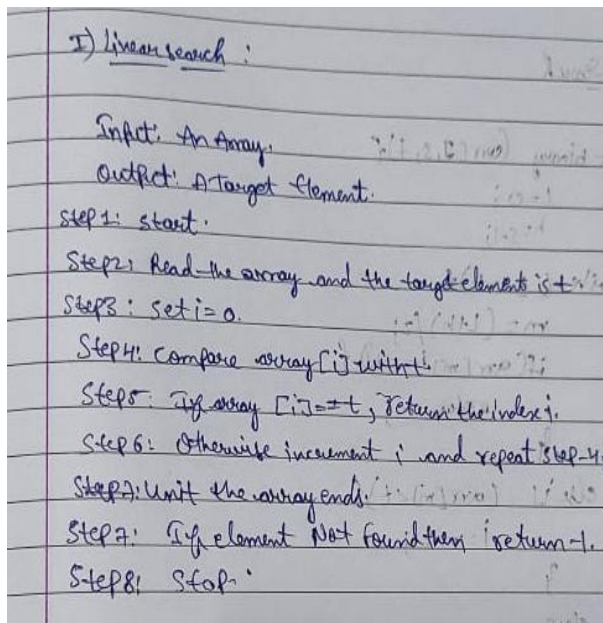


PRACTICAL: 02

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

LINEAR SEARCH

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int linearSearch(int array[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (array[i] == target) {
            return i;
        }
    }
    return -1;
}
```



```
}  
  
int main() {  
    int array[] = {1, 3, 2, 5, 6};  
    int n = sizeof(array) / sizeof(array[0]);  
    int target = 6;  
    cout << "Array is: ";  
    for (int i = 0; i < n; i++) {  
        cout << array[i] << " ";  
    }  
    int result = linearSearch(array, n, target);  
    cout << "\nTarget element: " << target << endl;  
    if (result != -1) {  
        cout << "Linear Search: Element found at index " << result << endl;  
    } else {  
        cout << "Linear Search: Element not found" << endl;  
    }  
    return 0;  
}
```

Output:

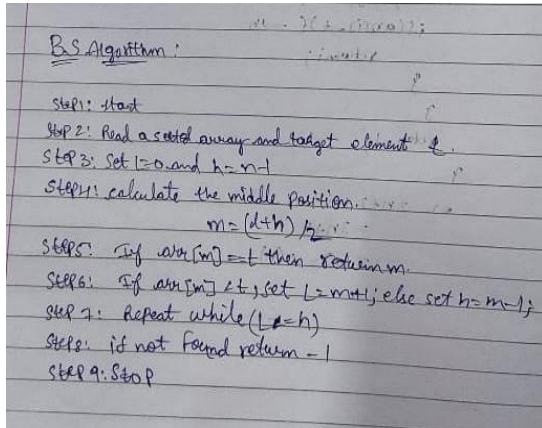
```
Array is: 1 3 2 5 6  
Target element: 6  
Linear Search: Element found at index 4
```

```
=== Code Execution Successful ===
```




BINARY SEARCH

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int binarySearch(int array[], int n, int target) {
    int low = 0;
    int high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (array[mid] == target) {
            return mid;
        }
        else if (array[mid] < target) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
}
```



```
    return -1;
}

int main() {
    int array[] = {1, 2, 3, 5, 6};
    int n = sizeof(array) / sizeof(array[0]);
    int target = 6;
    cout << "Array is: ";
    for (int i = 0; i < n; i++) {
        cout << array[i] << " ";
    }
    int result = binarySearch(array, n, target);
    cout << "\nTarget element: " << target << endl;
    if (result != -1) {
        cout << "Binary Search: Element found at index " << result << endl;
    } else {
        cout << "Binary Search: Element not found" << endl;
    }
    return 0;
}
```

Output:

```
Array is: 1 3 2 5 6
Target element: 6
Linear Search: Element found at index 4
```

=== Code Execution Successful ===



Time complexity:

```
ii) Binary Search
int binary (arr, l, s, t) {
    l = 0;
    h = s-1;
    while (l <= h) {
        m = (l+h)/2;
        if (arr[m] == t)
            return m;
        else if (arr[m] < t)
            l = m+1;
        else
            h = m-1;
    }
}
```

(i) Best Case = $O(1)$
 $mid = (low + high)/2$
If $(arr[mid] == key) \rightarrow 1$
 $T(n) = 1 + 1 = 2$
 $T(n) = O(1)$

(ii) Average Case = $O(\log n)$
 $n \rightarrow n/2$ take $\rightarrow n/4$...
no. of divisions $> \log_2 n$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

(iii) Worst Case = $O(\log n)$
 $n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \dots 1$
If $T(n) = \log_2 n$
 $T(n) = O(\log n)$

Conclusion:

Binary Search is a faster searching technique that repeatedly divides a sorted array into two halves to find the target element. Its time complexity is $O(\log n)$, making it more efficient than Linear Search for large sorted arrays.